

Problem Statement

ABC has successfully built the core backend APIs for its Online Shopping Application in the previous week. The next step is to enhance the application into a more secure, scalable, and enterprise-ready API platform using Python FastAPI. The business wants:

- secure access to customer accounts
- controlled admin actions
- reliable payment and order workflows
- better observability for failures
- non-blocking processing for selected I/O-heavy operations
- testable and maintainable code for future scale

Participants, this week, will enhance the Week 2 Online Shopping FastAPI application with JWT authentication, RBAC/ABAC, admin APIs, external integrations, async API development, background processing, logging, debugging, error handling, and testing.

The objective is to enhance the FastAPI Online Shopping Application by implementing:

- secure login and protected APIs
- role-based access control for different users
- policy-based authorization concepts
- external API integration patterns
- logging, monitoring, and debugging support
- async API development using FastAPI async/await patterns
- converting selected synchronous endpoints to asynchronous endpoints
- async DB session handling and async ORM usage patterns
- background task implementation for non-blocking operations
- understanding when to use async vs sync based on trade-offs
- unit and integration testing

Use the following concepts to build the solution to the given case study:

- Auth & OAuth2 / JWT / JWKS / OIDC concepts
- RBAC / ABAC with Casbin or equivalent policy logic
- decorators / reusable auth checks
- dependency injection with `Depends()`
- third-party API integration patterns
- retries, timeouts, circuit breaker thinking
- converting sync FastAPI endpoints to async
- async / await usage in API routes and service calls
- async DB sessions using context-managed patterns
- async ORM / repository access patterns
- background tasks
- async vs sync trade-offs
- structured logging



- custom exception handling
- debugging practices
- unit testing
- integration testing
- mocking dependencies
- coverage reporting
- linting and code quality practices
- basic dev workflow quality practices such as branch-based development, merge hygiene, and conflict resolution awareness

Functional Requirements

Implement the following requirements as part of developing the solution to this case study-

- **Secure Authentication-** Extend the Week 2 login flow by adding:
 - JWT-based authentication
 - Protected endpoints for cart and order operations
 - Token-based user identification for API access
 - Auth middleware or dependency-based route protection
- **Authorization-** Implement access rules:
 - Customer can browse products, manage cart, and place orders
 - Admin can manage product and category data
 - Support/Operations can view orders for troubleshooting
 - Enforce role-based or policy-based checks using reusable authorization logic
- **Product and Order Admin Operations-** Add controlled admin capabilities:
 - Create product
 - Update product
 - Deactivate or remove product
 - Create/update category
 - Access restricted to authorized roles only
- **External API Integration-** Introduce at least one external integration pattern, for example:
 - Payment gateway simulation
 - Shipment/tracking simulation
 - Notification/email API simulation
- **Logging, Monitoring & Error Handling-** Enhance the API with:
 - Structured request logs
 - Exception handling
 - Traceable error responses
 - Debug-friendly application behavior
 - Monitoring-oriented logs for operational visibility
- **Async API Development and Background Tasks-** Use async programming where appropriate and demonstrate clear implementation choices:

- Convert at least one existing synchronous FastAPI endpoint to an asynchronous endpoint
 - Use asynchronous database session handling for selected flows
 - Apply async ORM or repository access patterns where relevant
 - Implement background tasks for email notifications, audit log generation, or order confirmation steps
 - Ensure non-blocking behavior for I/O-heavy operations such as external service calls
 - Document when async should be preferred over sync and where sync is still acceptable
- **Testing**- Expand testing beyond basic validation by including:
 - Unit tests
 - Integration tests
 - Coverage validation for major business flows
 - Validation of security, integration, and async-enabled flows

Modules in Scope

Customer APIs

- Register
- Login
- Browse/search products
- View product details
- Manage cart
- Checkout
- Order history

Admin APIs

- Add product
- Update product
- Manage categories
- View order summaries
- Restricted through role/policy checks

System/Operational Features

- Audit logging
- External payment/shipping call
- Async order processing for selected workflows
- Async DB-backed transaction or order update scenario
- Background notification task
- Test suite for key flows

Suggested Application Layers

The case study expects layered architecture with API, business logic, and data access layers . For FastAPI, use this structure:

- API Layer
- FastAPI routers/endpoints
- Service Layer
- Business logic and validations
- Repository Layer
- Database interaction methods
- Data Layer
- SQLAlchemy models + Postgres
- Schema Layer
- Pydantic request/response models

API Enhancements

Authentication

- `POST /api/auth/login`
- `POST /api/auth/refresh` (optional)
- protected routes using bearer token

Customer APIs

- `GET /api/products`
- `GET /api/products/{id}`
- `GET /api/products/search`
- `POST /api/cart/add`
- `PUT /api/cart/update/{cart\_item\_id}`
- `DELETE /api/cart/remove/{cart\_item\_id}`
- `POST /api/orders/checkout`
- `GET /api/orders/me`

Admin APIs

- `POST /api/admin/products`
- `PUT /api/admin/products/{id}`
- `POST /api/admin/categories`
- `PUT /api/admin/categories/{id}`
- `GET /api/admin/orders`

Operational/Integration APIs

- `POST /api/payments/process`
- `POST /api/notifications/send` (or background task trigger)

Authorization Design

Use role/policy rules such as:

Customer	Browse products, manage cart, place/view own orders
Admin	Manage products and categories, view all orders
Support	View orders and customer issues, but no product admin rights

Async Implementation Expectation

Participants must identify at least one existing synchronous business flow and refactor it to an asynchronous implementation. Suggested examples include:

- checkout flow
- payment processing flow
- notification dispatch after order creation
- admin order lookup with external dependency interaction

The implementation should demonstrate:

- async route handling
- async service or repository interaction where applicable
- async DB session usage for selected flows
- non-blocking external API handling
- appropriate use of background tasks
- justification for choosing async or sync for the selected flow

Validations

Extend Week 2 validations using the following:

Security Validations

- Protected endpoints require valid token
- Expired/invalid token should return proper error
- Restricted routes must enforce role/policy checks

Admin Validations

- Only authorized roles can create/update products
- Duplicate product/category checks
- Invalid admin input should be rejected clearly

Integration Validations

- External API failures should be handled gracefully
- Timeout/retry logic should not crash order flow
- Order/payment status must remain consistent

Async API Validations

- At least one endpoint must be refactored from sync to async
- Async implementation should be used only where it improves I/O handling or responsiveness
- Selected DB interactions should demonstrate async session usage
- Background task execution should not block the main API response
- Participants must explain why async was chosen for the implemented flow and where sync remains acceptable

Logging & Debugging Validations

- Request and error logs should support traceability
- Failures should be diagnosable through logs and error responses
- Async operations should produce readable and traceable logs

Testing Validations

- Major success and failure paths should be covered
- Mock dependencies for repeatable tests
- Tests should include validation of security, integration, and selected async scenarios

Technology Stack

Framework	Python Fast API
Programming Language	Python 3.12
IDE	Visual Studio Code
Data Source	SQLite and PostgreSQL

Non-Functional Requirements

The solution calls for low latency, high throughput, scalability, maintainability, and testability.

- Secure API design
- Reusable authorization components
- Readable logs and diagnosable failures
- Reliable external integration handling
- Production-oriented code structure
- Automated test coverage on key flows
- Appropriate use of async patterns without unnecessary complexity
- Maintainable separation of sync vs async responsibilities

Guidelines

- Follow best coding practices.
- Use proper folder structure and naming conventions.

</>

- Keep the design extensible for future modules such as admin features.
- Add comments wherever useful, for understanding.
- Make reasonable assumptions if some details are not explicitly mentioned.
- Focus on working functional API over UI or advanced security.
- Use Git to maintain project repository, manage pull requests, and resolve conflicts.

Milestones

Milestone#	Milestone Description
Milestone 1	Implement JWT-based authentication and protected routes
Milestone 2	Implement role-based / policy-based authorization with reusable access checks
Milestone 3	Implement admin product, category, and order access APIs
Milestone 4	Implement external API integration with retry and timeout handling
Milestone 5	Implement async processing, background tasks, logging, and exception handling
Milestone 6	Add unit testing, coverage validation, and finalize Git-based delivery